

天津大学

《机器学习综合实践》课程报告



基于 PatchCore 的工业图像异常检测

学 号_

姓 名_

邮 箱_____

学 院_____ 智算学部

专 业_____ 人工智能

年 级_____ 23

任课教师_____ 王煜

2025 年 9 月 16 日

摘 要

本报告围绕工业产品图像异常检测任务，研究并实现了基于 PatchCore 的无监督异常检测方法，解决传统检测方法适应性差及深度学习全监督方法异常样本稀缺的问题。

报告首先阐述实验背景与意义，工业缺陷检测对产品质量与生产安全不可或缺，选择无监督 PatchCore 方法的原因。实验采用 MVTecAD 数据集子集，目标为图像级异常检测，以 AUROC 为评估指标。

在方法部分，详细介绍 PatchCore 算法流程：通过 ImageNet 预训练模型提取中层特征，经邻域聚合与步幅采样构建局部 Patch 特征，融合多层级特征后形成正常样本 Patch 特征记忆库；引入核心集采样，在保留正常特征分布的同时降低记忆库冗余，提升推理效率；测试阶段通过计算测试 Patch 与记忆库核心集的欧氏距离，结合重加权策略得到图像级异常分数，实现异常判定。

实验与扩展部分通过多组实验，确定最优的参数：预训练模型优选 WideResNet101、特征层级采用 layer2+3、核心集采样比例 1%-30%、Patch 邻域大小 3×3 。在此配置下，在 Kaggle 平台上得到了图像级 AUROC 98.93 的成绩，进一步实现像素级异常定位，标注异常区域。

还分析了 PatchCore 现存问题，如特征域偏移、多尺度异常捕捉能力弱、kNN 搜索效率瓶颈，并提出改进方向。

关键词：异常检测，图像级，PatchCore

目 录

第一章	引言	1
1.1	实验背景和意义	1
1.2	实验目标与内容	2
1.2.1	数据集	2
1.2.2	训练目标	3
第二章	方法	4
2.1	为每个类别单独建模	4
2.2	相关方法	4
2.3	获取局部 Patch 特征	5
2.3.1	符号定义	5
2.3.2	预训练模型输出特征层级选择	5
2.3.3	构建局部 patch 特征	5
2.3.4	代码实现	6
2.4	核心集采样	13
2.4.1	内存库规模过大	13
2.4.2	选择核心集	13
2.4.4	代码实现	15
2.5	实现异常检测	17
2.5.1	计算图像级异常评分	17
2.5.2	代码实现	18
2.6	总结	23
第三章	实验与扩展	23
3.1	模型参数分析	23
3.1.1	预训练模型的影响	24
3.1.2	提取特征图层级的影响	24
3.1.3	下采样百分比	25
3.1.4	Patch 邻域大小	26
3.1.5	总结	27
3.2	推理时间分析	28
3.3	少量样本异常检测实验	29
3.4	提高总体 AUROC 的小技巧	30
3.5	像素级定位实现	30
3.5.1	代码实现	33
第四章	开放性思考与讨论	36
4.1	特征域偏移	36
4.2	多尺度异常捕捉能力不足	36
4.3	计算效率低	37

第五章 结论.....	38
参考文献.....	38

第一章 引言

1.1 实验背景和意义

随着互联网技术的持续发展，计算机视觉作为该领域的重要分支，在社会生产活动中的作用日益凸显，并逐渐成为推动技术进步和产业升级的关键力量，并被广泛应用于多个高精度领域。在当下高度自动化的工业生产环境中，产品质量的稳定性直接关系到企业的市场竞争力和经济效益。工业制品的缺陷不仅影响产品的外观和性能，还可能导致严重的安全问题。因此，工业产品缺陷检测技术在工业生产过程中的重要性不言而喻。作为计算机视觉的其中一个重要的应用场景，图像异常检测技术在实际应用中能够高效识别异常情况，实现对问题产品的快速侦测，从而提升生产流程的自动化水平、优化产品质量控制体系，对智能制造和精细化管理具有深远影响。

图像异常检测是指在产品生产过程中，通过计算机视觉的技术和方法，对产品或材料的表面及内部进行检测，以发现其中存在的缺陷、瑕疵、异常等问题。因此，提出高效的缺陷检测方法广泛受到国内外研究者的关注。

在工业缺陷检测领域，传统方法主要依赖于手工特征提取和经典图像处理技术，然后检测对象的特征与规则或模板进行匹配，判断是否存在异常，是一种依赖于预定义的标准或模板进行判断的方法。然而，这种方法存在相当的局限性，它依赖固定的规则、模板或接触式测量设备，难以适应复杂、多变的检测需求，此外，覆盖范围较窄，难以处理复杂或未知的异常模式，且需要依赖专家知识制定规则，耗时较长，预算较高，无法满足高效生产的需求。

近年来，基于深度学习的异常检测技术已成为该领域的研究热点。因其能够处理高维、复杂数据，并自动提取数据特征，已经广泛应用于工业检测、金融分析、医疗诊断和网络安全等多个领域。这类技术主要通过深度神经网络学习正常数据的分布模式，从而识别与之偏离的异常数据点。研究者们提出了多种基于深度学习的缺陷检测方法，包括全监督学习、无监督学习和半监督学习。

其中，全监督学习方法在工业缺陷检测中十分重要，旨在通过大规模标注数据集训练模型，以实现高精度的缺陷识别与定位，但是，使用这种方法需要同时含有正、负例的数据集进行训练，实际场景中异常样本稀缺且难以获取，同时异常数据具有不确定性、稀有性和多样性等特点，导致有监督学习方法面临数据不足、特征复杂和类别不平衡等挑战，具体来说，异常样本的稀缺性导致训练数据难以覆盖所有异常模式，同时异常数据的复杂特征与随机分布特性使得人工标注过程缺乏统一标准与客观依据。

于是，无监督训练方式提出，这一方法不依赖于标签数据，已成为异常检测领域的重要发展方向，并取得了显著的研究成果。无监督训练包括基于重建的方法以及基于表示的方法。

基于重建的方法通常采用图像生成器，例如自动编码器，生成对抗网络，扩散模型等，来生成或重建输入图像，并通过输入图像与重建图像之间的差异来定位异常区域，如图 1-1 所示。这些方法对正常图像进行建模，并进行重建，他们的假设是，由于异常在训练过程中未被观察到，因此无法对其进行重建。在推理阶段，当重建图像与原始图像出现偏差时，就会发现异常区域。

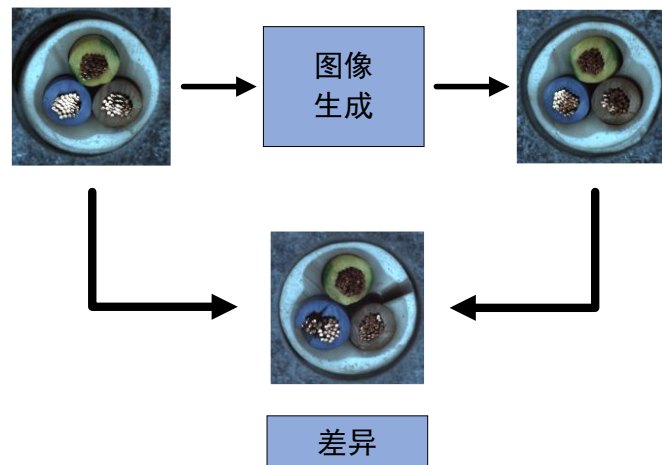


图 1-1 基于重建的异常检测方法的流程

基于表示的方法采用了图 1-2 中的方法，它首先使用不同的特征提取器来获取正常图像或正常图像块的判别性特征，然后建立这些正常特征分布。最后，通过计算测试图像的嵌入与正常图像表示分布之间的距离来计算异常分数。因此，特征提取器、分布估计器和距离度量是该方法中的三个主要组成部分。对于特征提取器，现有方法通常采用基于 CNN 或基于 ViT 的模型，对于分布估计器，正常图像的分布通常是正常图像的 n 维球面中心、正常图像的高斯分布，或者整个正常图像嵌入的 k 近邻（kNN）为特征。我们使用的 PatchCore 方法使用一个由正常图像块特征组成的最具代表性的记忆库，并采用 k 近邻方法来搜索最接近的特征。

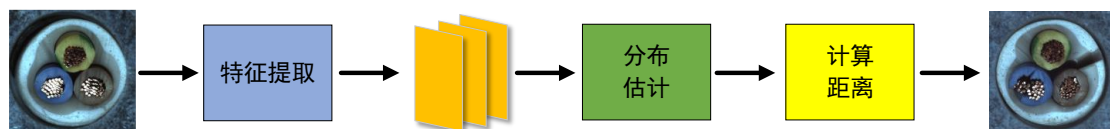


图 1-2 基于表示的异常检测方法的流程

最后，半监督的训练方法，将全监督学习与无监督学习的优势特性相融合，形成半监督检测机制，仅需部分标签数据和大量无标签数据参与训练的模式，有效缓解了缺陷样本稀缺的难题，但受限于异常特征学习不充分。

1.2 实验目标与内容

1.2.1 数据集

MVTec AD（MVTec Anomaly Detection）是由 MVTec Software GmbH 推出的面向无监督异常检测任务的综合真实世界数据集，为该领域提供高质量、贴近实际应用的数据资源，为计算机视觉领域异常检测方法的研发与评估提供有力支撑。

该数据集为无监督异常检测下的场景，模拟了模拟工业视觉检测的真实需求，

即训练阶段仅有无缺陷样本，测试阶段则包含了多种缺陷的样本，并提供像素级的异常区域定位，包含 5354 张高分辨率彩色图像，分为 15 个类别（5 类纹理 + 10 类物体）。

为了更快的验证模型的性能，以及更好的适应较低性能的场景，我们使用该数据集的子集，MVTec-AD_subset 进行异常检测的任务，具体来说，我们只使用 cable、capsule、metal_nut、screw、transistor 这几个类别，由图 1-3 所示，进行图像级的异常检测任务，其中，训练集共 1196 张图片，验证集共 50 张图片，测试集共 657 张图片。train.csv: 训练集图片标注，0 为正常，1 为异常；val.csv: 验证集图片标注；test.csv: 测试集图片标注。

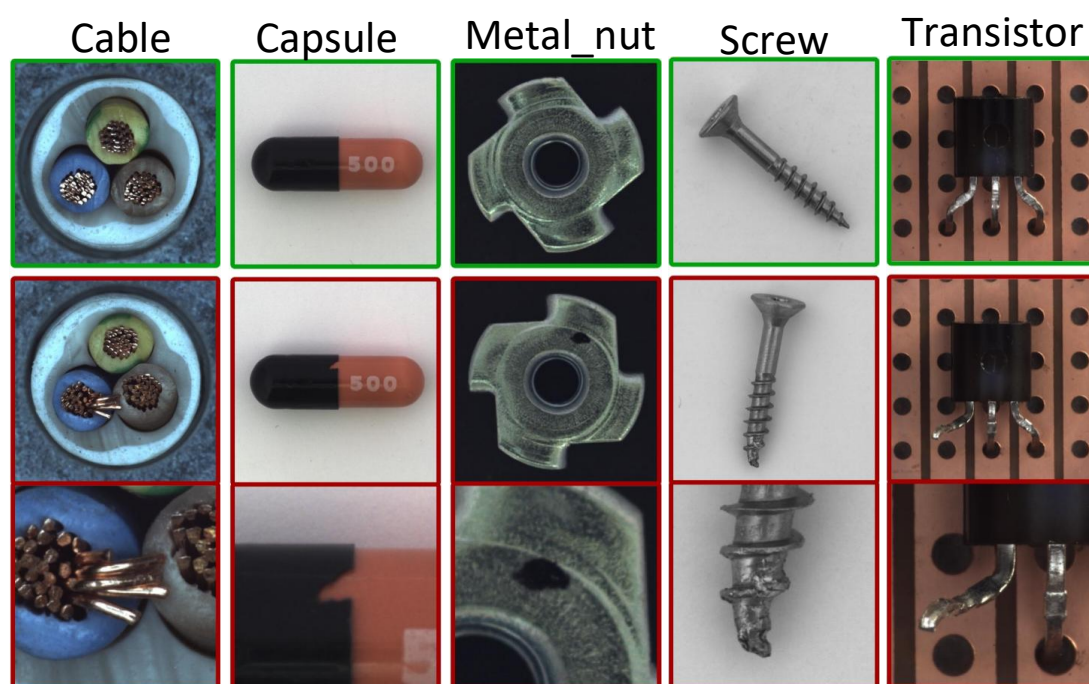


图 1-3 MVTec AD 子集中的样例图片

1.2.2 训练目标

异常检测共分为两个级别：图像级，像素级。我们的训练目标是，进行图像级别的异常检测，具体来说，就是检测整张图片是异常图片还是正常图片，这是一个二分类的任务，因此，我们的评估指标为 AUROC（受试者工作特征曲线下面积），尤其适用于二分类的模型性能评估。

二分类模型通常输出一个概率值，而不是直接的正/负标签。我们需要设定一个阈值，当分数大于阈值时，就会判定该图片为异常图片。而为了计算 AUROC，我们不需要进行最后阈值的设定，而是只依靠异常分数，就可以计算完成，因为它本质上是对模型在所有可能阈值下的综合性能评估，计算 AUROC 时，会遍历并整合所有潜在阈值对应的分类效果，最终形成一个独立于具体阈值的指标。

在进行计算时，首先将所有样本按照模型输出的异常分数（概率值）从高到低排序。随后，每一个样本的分数都会被依次当作假定阈值——即认为分数大于等于该值的样本为异常，小于该值的为正常，然后针对每一个假定阈值，都会

计算出一组对应的真阳性率（TPR），即异常样本被正确识别的比例和假阳性率（FPR），即正常样本被错误识别为异常的比例。这两个指标构成了 ROC 曲线上的一个点，最后，所有样本都被作为阈值遍历后，所有（FPR,TPR）点会连接成完整的 ROC 曲线。而 AUROC 就是这条曲线下方的面积，因此无需依赖某个特定的阈值。

第二章 方法

我们的异常检测方法使用基于 PatchCore 的方法，在性能方面达到优秀，并且在效率方面也有较大优势，在本章，我们将阐述该方法的算法原理与具体实现。

对于 MVTec-AD_subset，我们使用给每一个类别都单独训练的方法，在 2.1 节中，进行解释；2.2 节介绍了与 PatchCore 相关的方法；2.3,2.4,2.5 介绍了 PatchCore 方法的 3 个组成部分，分别为，提取 Patch 特征并加入记忆库，可以提高效率的核心集采样方法，最终得出决策的完整算法。

2.1 为每个类别单独建模

在 MVTec-AD_subset 上训练有两种不同的方式，一种是给每一个类别单独训练一个模型，另一种是将所有类别的图像混合后训练一个通用模型，虽然数据是所有类别放在一个 train 目录下，但我们选择使用前者训练方式，原因如下：

（1）不同类别的物体在外观、缺陷表现等方面存在较大差异，例如电缆的缺陷一般为导线弯曲、外部绝缘层破损等，而螺丝钉的缺陷一般为划痕、螺纹受损等，如果每个类别单独训练，那么模型只需要区分是否有异常，如果一块训练，那么模型很难同时学习到这么多差异极大的模式，导致混淆。

（2）分类别训练能让模型专注于学习某一类对象的正常特征，提高学习效率，可以更精准地捕捉到特定零部件的细节特征，能够更敏感地发现与正常特征的差异。如果所有样本一起训练，模型需要学习的特征范围太广，难以深入学习每个类别的关键特征，可能会遗漏一些细微但重要的正常特征信息，影响异常检测效果。

（3）在实际应用中，往往是针对特定类型的产品进行异常检测，比如生产线上专门检测某一种手机屏幕的缺陷。分类别训练得到的模型此时具有很强的针对性，方便快捷。

（4）现有的基于 MvTec-AD 数据集的研究多针对分类别训练，在学术界已经形成共识。

2.2 相关方法

有研究表明[3]，使用在 ImageNet 等大型图像数据集上预训练的模型，无需对工业图像数据进行适配，即可实现较高的异常检测性能，这一发现催生了很多基于预训练模型的工业异常检测方法，如 SPADE[4]，该方法使用预训练网络的深层特征表示，受 ImageNet 偏见影响大，导致对工业场景适配性差，而 PatchCore 方法则采用中层级的特征表示，在保持较高分辨率的同时，最大限度减少对 ImageNet 的偏向。

PaDiM[5]把图像拆解为多个局部 Patch，并针对每个 Patch 提取对应的特征

表示。对于正常样本的这些 Patch 特征，PaDiM 会计算均值和协方差，以此构建出正常数据在 Patch 特征空间中的分布。在测试阶段，对于测试样本的每个 Patch 特征，PaDiM 计算该特征与正常样本对应的 Patch 特征分布之间的马氏距离，来衡量其偏离正常分布的程度，这种方法依赖于每个 Patch 特定的马氏距离，对图像对齐要求较高，且可利用的上下文范围有限，而 PatchCore 构建了可被测试时所有 Patch 访问的更高效记忆库，能利用更广泛的正常上下文进行异常估计，同时 PatchCore 不要求训练与测试图像具有相同形状，减少了对图像对齐的依赖。

2.3 获取局部 Patch 特征

2.3.1 符号定义

用 $\mathcal{X}_N$ 表示训练阶段的所有正常样本图像集合，其中每个图像 $x \in \mathcal{X}_N$ 的标签为 $y_x = 0$ ，测试集为 $\mathcal{X}_T$ ，标签为 $y_x \in \{0,1\}$ ，其中 0 为正常，1 为异常。采用在 ImageNet 上预训练的模型 ϕ 作为特征提取器，用 $\phi_{i,j} = \phi_j(x_i)$ 表示图像 x_i 在预训练模型的层级 j 上的输出特征图，对于 ResNet 系列模型， $j \in \{1,2,3,4\}$ 对应不同层级的残差块的输出。

2.3.2 预训练模型输出特征层级选择

直接使用网络最后一层，或者高层级的特征图存在两个问题，第一，高层特征抽象度高，缺少划痕、细微凹陷等这些细节异常，而测试阶段的异常类型往往是未知的，这种缺失会影响模型性能。第二，高层特征是为自然图像分类任务优化的，与工业场景中检测部件结构，表面平整等这些内容的适配性差。

因此，PatchCore 选择中层特征，例如 ResNet 模型的 $j \in \{2,3\}$ 层级，构建 patch 特征，避免了浅层特征的噪声干扰，又减少了 ImageNet 任务带来的偏向。

2.3.3 构建局部 patch 特征

假设特征图 $\phi_{i,j} \in \mathbb{R}^{C \times H \times W}$ 是一个通道为 C ，高 H ，宽 W 的特征图，那么用

$\phi_{i,j}(h, w) \in \mathbb{R}^C$ 来表示在位置 $h \in \{1,2,\dots,H\}$ ， $w \in \{1,2,\dots,W\}$ 上的 C 维特征切片，因为特征提取器的感受野大于 1，因此，这个特征切片就对应了图像中的一个 Patch 特征。扩大单个 Patch 特征的感受野，可以使用池化或者使用深层特征来实现，但是这会使得 patch 特征会变得更具有 ImageNet 偏向，因此，引入邻域聚合机制。

首先，定义一个邻域的范围，对特征图上的位置 (h,w) ，设定邻域大小 p ，他的邻域 $\mathcal{N}_p^{(h,w)}$ 是以 (h,w) 为中心，边长为 p 的正方形区域，假设 $p=3$ ，那么邻域就是 3×3 的正方形。然后对这个领域应用聚合函数，我们使用自适应平均池化，对邻域内所有特征向量进行聚合，生成单个位置 (h,w) 的局部感知特征 $\phi_{i,j}(\mathcal{N}_p^{(h,w)})$ ，

由公式 2-1 所示, f_{agg} 表示自适应平均池化, PatchCore 对特征图的所有位置都做一次邻域聚合, 这样既保留了特征图的分辨率, 又通过融合周边信息增强了对局部空间变化的鲁棒性。

$$\phi_{i,j}(\mathcal{N}_p^{(h,w)}) = f_{agg}(\{\phi_{i,j}(a,b) | (a,b) \in \mathcal{N}_p^{(h,w)}\}) \quad (2-1)$$

接着, 引入步幅参数 s 对聚合后的特征进行采样, 仅保留 h 和 w 均为 s 整数倍的位置对应的局部 patch 特征, 默认 $s=1$, 即保留全部特征。最终形成图像 x_i 在层级 j 的局部 patch 特征集合, 记为公式 2-2。

$$\mathcal{P}_{s,p}(\phi_{i,j}) = \{\phi_{i,j}(\mathcal{N}_p^{(h,w)}) | h, w \bmod s = 0, h < H, w < W, h, w \in \mathbb{N}\} \quad (2-2)$$

为了兼顾不同抽象程度的信息, PatchCore 融合相邻两个中层层级, 例如 j 和 $j+1$, 因为处于不同的层级, $j+1$ 的特征图分辨率要低于 j 的特征图, 因此, 先通过双线性插值将 $\mathcal{P}_{s,p}(\phi_{i,j+1})$ 缩放至与 $\mathcal{P}_{s,p}(\phi_{i,j})$ 具有相同规模, 然后将对应位置聚合, 形成最终的 patch 特征集合。

对所有训练阶段的正常样本 $x_i \in \mathcal{X}_N$, 提取其多层级融合后的 patch 特征集合, 合并后构成正常样本 patch 特征内存库 M , 存储了正常样本的 patch 特征分布, 为后续测试阶段的异常判断提供参考依据。

图 2-1 展示了构建局部 patch 特征的全部流程, 包括从预训练网络中提取特征图, 然后对特征图进行邻域聚合以及多级融合, 最终得到 patch 特征, 将训练集中所有的图片进行该操作, 即可得到记忆库 M 。

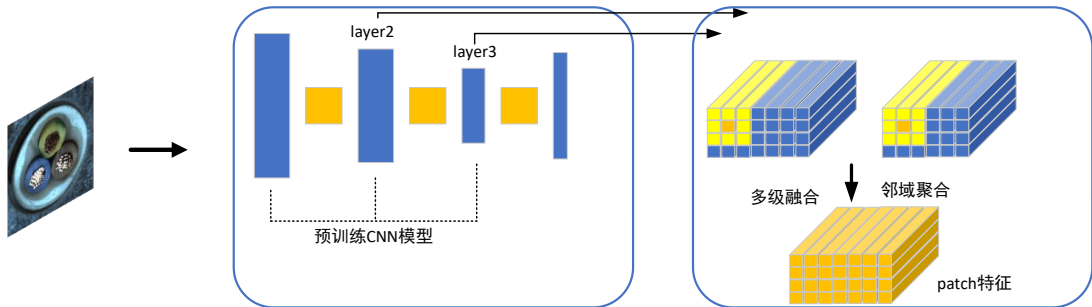


图 2-1 构建 patch 特征流程

2.3.4 代码实现

2.3.4.1 预训练网络特征提取

首先使用预训练网络的中层特征输出, 具体实现如代码 2-1 所示。

```
class NetworkFeatureAggregator(torch.nn.Module):
```

```
    """高效提取网络特征的模块。"""
```

```
    def __init__(self, backbone, layers_to_extract_from, device):
```

```
        super(NetworkFeatureAggregator, self).__init__()
```

"""网络特征提取初始化方法。

仅运行网络至需要提取特征的最后一层（即 `layers_to_extract_from` 列表的最后一个元素），避免冗余的后续层计算。

参数：

```

    backbone: torchvision.model
        预训练的主干网络（如 ResNet、WideResNet 等）
    layers_to_extract_from: [list of str]
        需要提取特征的网络层名称列表（如 ["layer2", "layer3"]）
    device: torch.device
        计算设备（CPU 或 GPU）
"""

self.layers_to_extract_from = layers_to_extract_from # 待提取特征的层列表
self.backbone = backbone # 主干网络
self.device = device # 计算设备
# 若主干网络无 hook_handles 属性，初始化为空列表（用于管理钩子句柄）
if not hasattr(backbone, "hook_handles"):
    self.backbone.hook_handles = []
# 移除已存在的钩子，避免重复注册
for handle in self.backbone.hook_handles:
    handle.remove()
self.outputs = {} # 用于存储各层提取的特征

# 为每个待提取特征的层注册前向钩子
for extract_layer in layers_to_extract_from:
    # 实例化钩子：传入特征存储字典、当前提取层名称、最后一个提取层名称
    forward_hook = ForwardHook(
        self.outputs, extract_layer, layers_to_extract_from[-1]
    )
    # 处理含层级结构的层名称（如 "layer2.3"，表示 layer2 的第 3 个子层）
    if "." in extract_layer:
        extract_block, extract_idx = extract_layer.split(".")
        # 获取主干网络中的对应块（如 backbone.layer2）
        network_layer = backbone.__dict__["_modules"][extract_block]
        # 若索引为数字，获取块中的序列层（如 layer2[3]）
        if extract_idx.isnumeric():
            extract_idx = int(extract_idx)
            network_layer = network_layer[extract_idx]
        # 若索引为字符串，获取块中的子模块（如 layer2.conv1）
        else:
            network_layer = network_layer.__dict__["_modules"][extract_idx]
    # 处理直接命名的层（如 "layer3"）
    else:

```

```

network_layer = backbone.__dict__["_modules"][extract_layer]

# 若目标层是 Sequential 序列，为序列的最后一个子层注册钩子
if isinstance(network_layer, torch.nn.Sequential):
    self.backbone.hook_handles.append(
        network_layer[-1].register_forward_hook(forward_hook)
    )
# 若目标层是单个模块，直接为该层注册钩子
else:
    self.backbone.hook_handles.append(
        network_layer.register_forward_hook(forward_hook)
    )
# 将模块移动到指定设备
self.to(self.device)

def forward(self, images):
    """前向传播：提取并返回指定层的特征。

    参数:
        images: torch.Tensor
            输入图像张量，形状通常为 [batch_size, channels, height, width]

    返回:
        self.outputs: dict
            键为层名称，值为对应层的特征张量
    """
    self.outputs.clear() # 清空上一轮的特征存储
    with torch.no_grad(): # 推理模式，禁用梯度计算
        # 主干网络前向传播，当到达最后一个提取层时会抛出异常以终止后续计算
        try:
            _ = self.backbone(images)
        except LastLayerToExtractReachedException:
            pass # 捕获异常，不执行额外操作
    return self.outputs # 返回提取的所有层特征

def feature_dimensions(self, input_shape):
    """计算给定输入形状下，各提取层的特征维度。

    参数:
        input_shape: tuple
            输入图像的形状（不含 batch 维度），如 (3, 224, 224)

    返回:
        list of int
    """

```

```

        各提取层的特征通道数（如 [512, 1024] 对应 layer2 和 layer3 的通道数）
"""
# 构造符合输入形状的虚拟张量（batch_size=1）
_input = torch.ones([1] + list(input_shape)).to(self.device)
# 前向传播获取特征
_output = self(_input)
# 提取各层特征的通道数（特征张量形状通常为 [1, c, h, w]，取索引 1 即通道数 c）
return [_output[layer].shape[1] for layer in self.layers_to_extract_from]

```

代码 2-1 提取预训练特征实现

实现了 NetworkFeatureAggregator 模块，通过为目标层注册前向钩子，在网络前向传播时自动捕获并存储指定层的特征，避免冗余的后续层计算。当到达最后一个目标提取层时，会通过异常终止传播以提升效率。

2.3.4.2 patch 生成

class PatchMaker:

```

"""patch 工具类：实现特征图的 patch 化与分数反 patch 化"""
def __init__(self, patchsize, stride=None):
    self.patchsize = patchsize # patch 大小（默认 3，即 3x3 patch）
    self.stride = stride # patch 滑动步长（默认 1，密集重叠 patch）

def patchify(self, features, return_spatial_info=False):
    """
    将特征图转换为 patch 张量
    Args:
        features: 输入特征图，shape 为[批量大小x通道数x高x宽]
        return_spatial_info: 是否返回 patch 的空间尺寸信息
    Returns:
        patch 特征（shape: [批量xpatch 总数x通道xpatch 高xpatch 宽]）+ 可选的空间尺寸
    """
    # 计算填充大小: 保证 patch 中心与特征图像素对齐(patch 大小为奇数时, 填充为(patchsize-1)/2)
    padding = int((self.patchsize - 1) / 2)
    # 定义滑动窗口展开器: 按指定 patch 大小、步长、填充切分特征图
    unfold = torch.nn.Unfold(
        kernel_size=self.patchsize, stride=self.stride, padding=padding, dilation=1
    )
    # 展开特征图: shape 转为[批量x(通道xpatch 高xpatch 宽)xpatch 总数]
    unfolded_features = unfold(features)
    # 计算每个维度的 patch 数量（高方向和宽方向）
    number_of_total_patches = []
    for s in features.shape[-2:]: # 遍历特征图的高和宽
        # patch 数量计算公式: (特征尺寸 + 2x填充 - patch 大小) / 步长 + 1
        n_patches = (
            s + 2 * padding - 1 * (self.patchsize - 1) - 1
        ) / self.stride + 1
    
```

```

        number_of_total_patches.append(int(n_patches))
# 重塑 patch 特征: 转为[批量×通道×patch 高×patch 宽×patch 总数]
unfolded_features = unfolded_features.reshape(
    *features.shape[:2], self.patchsize, self.patchsize, -1
)
# 调整维度顺序: 转为[批量×patch 总数×通道×patch 高×patch 宽] (符合后续处理习惯)
unfolded_features = unfolded_features.permute(0, 4, 1, 2, 3)

# 若需要, 返回 patch 特征和空间尺寸; 否则仅返回 patch 特征
if return_spatial_info:
    return unfolded_features, number_of_total_patches
return unfolded_features

def unpatch_scores(self, x, batchsize):
    """将扁平化的 patch 分数重塑为带批量维度的形状 (用于后续定位映射)"""
    return x.reshape(batchsize, -1, *x.shape[1:])

```

代码 2-2 将特征图转换为 patch

```

def _embed(self, images, detach=True, provide_patch_shapes=False):
    """返回图像的特征嵌入"""

    def _detach(features):
        # 若 detach 为 True, 将特征从计算图分离并转为 numpy 数组 (减少 GPU 占用, 便于后续处理)
        if detach:
            return [x.detach().cpu().numpy() for x in features]
        return features

    # 特征提取器设为评估模式 (无需微调预训练骨干网络)
    _ = self.forward_modules["feature_aggregator"].eval()
    # 关闭梯度计算 (非训练阶段, 提升效率)
    with torch.no_grad():
        # 用预训练骨干网络 (如 WideResNet50) 提取图像特征
        features = self.forward_modules["feature_aggregator"](images)

    # 仅保留指定的中间层特征 (如 ResNet 的 2、3 层, 避免高层特征的 ImageNet 偏置)
    features = [features[layer] for layer in self.layers_to_extract_from]

    # 对每个层级的特征图进行 patch 化处理, 同时返回 patch 的空间尺寸信息
    features = [
        self.patch_maker.patchify(x, return_spatial_info=True) for x in features
    ]
    # 提取各层级特征对应的 patch 空间尺寸 (如(h_patches, w_patches))
    patch_shapes = [x[1] for x in features]
    # 提取 patch 化后的特征 (剥离空间尺寸信息)

```

```

features = [x[0] for x in features]
# 以第一个层级的 patch 尺寸为参考标准（用于后续对齐不同层级特征）
ref_num_patches = patch_shapes[0]

# 对齐不同层级特征的 patch 尺寸（因不同层特征图分辨率不同）
for i in range(1, len(features)):
    _features = features[i] # 当前层级的 patch 特征
    patch_dims = patch_shapes[i] # 当前层级的 patch 空间尺寸

    # 重塑特征形状：适配后续插值操作，将 patch 维度与空间维度分离
    _features = _features.reshape(
        _features.shape[0], patch_dims[0], patch_dims[1], *_features.shape[2:]
    )
    # 调整维度顺序：将空间维度后置，便于插值
    _features = _features.permute(0, -3, -2, -1, 1, 2)
    perm_base_shape = _features.shape # 记录当前维度结构（用于后续恢复）
    # 展平特征：适配双线性插值的输入格式
    _features = _features.reshape(-1, *_features.shape[-2:])
    # 双线性插值：将当前层级 patch 尺寸对齐到参考尺寸
    _features = F.interpolate(
        _features.unsqueeze(1), # 增加通道维度（插值函数要求输入为 4D）
        size=(ref_num_patches[0], ref_num_patches[1]), # 目标尺寸（参考层级的 patch
尺寸）
        mode="bilinear", # 双线性插值模式
        align_corners=False,
    )
    # 移除多余的通道维度
    _features = _features.squeeze(1)
    # 恢复特征维度结构：将插值后的特征还原为原始的 patch 特征格式
    _features = _features.reshape(
        *_perm_base_shape[:-2], ref_num_patches[0], ref_num_patches[1]
    )
    # 调整维度顺序：恢复为“样本数×patch 数×通道×patch 高×patch 宽”格式
    _features = _features.permute(0, -2, -1, 1, 2, 3)
    _features = _features.reshape(len(_features), -1, *_features.shape[-3:])
    features[i] = _features # 更新对齐后的特征
# 展平特征：将“样本数×patch 数×通道×patch 高×patch 宽”转为“(样本数×patch 数)×通道×patch
高×patch 宽”
features = [x.reshape(-1, *x.shape[-3:]) for x in features]

# 预处理：统一不同层级特征的维度
features = self.forward_modules["preprocessing"](features)
# 聚合多层级特征（如拼接 2 层和 3 层特征）
features = self.forward_modules["preadapt_aggregator"](features)

```

```
# 若需要, 返回 patch 特征和 patch 空间尺寸; 否则仅返回 patch 特征
if provide_patch_shapes:
    return _detach(features), patch_shapes
return _detach(features)
```

代码 2-3 转换 patch 主函数

然后将图像转换为标准化的补丁，这两段代码中，`_embed` 方法是特征嵌入的主函数，接收图像输入，输出标准化的补丁特征，具体流程为：初始化预训练的特征提取模型，得到中间层特征，接下来调用 `Patchify` 方法，将特征图进行初始的 patch 化，具体就是使用滑动窗口来切分特征图，通过双线性插值将高层特征与低层特征对齐，标准化不同层级特征的通道数，使用自适应平均池化来进行聚合，输出 patch 特征。

2.3.4.3 构建记忆库

```
def _fill_memory_bank(self, input_data):
    """计算并设置支持特征，用于构建记忆库"""
    # 将前向传播模块设为评估模式（无需训练，仅用于特征提取）
    _ = self.forward_modules.eval()

    def _image_to_features(input_image):
        # 关闭梯度计算，避免不必要的资源消耗（特征提取阶段无参数更新）
        with torch.no_grad():
            # 将输入图像转为浮点型并迁移到指定设备（如 GPU）
            input_image = input_image.to(torch.float).to(self.device)
            # 调用 _embed 方法提取图像的 patch 特征（返回预处理后的特征嵌入）
            return self._embed(input_image)

    # 初始化列表用于存储所有输入数据的特征
    features = []
    # 使用 tqdm 创建进度条，显示特征计算进度（位置 1，不保留进度条痕迹）
    with tqdm.tqdm(
        input_data, desc="Computing support features...", position=1, leave=False
    ) as data_iterator:
        # 遍历输入数据中的每张图像
        for image in data_iterator:
            # 若图像以字典形式存储，提取"image"字段对应的图像数据
            if isinstance(image, dict):
                image = image["image"]
            # 计算当前图像的特征并添加到列表中
            features.append(_image_to_features(image))

    # 将所有图像的特征在第 0 维（样本维度）拼接，形成完整的特征集合
```

```
features = np.concatenate(features, axis=0)
# 对特征集合进行采样（如核心集采样），减少冗余并优化记忆库规模
features = self.featuresampler.run(features)

# 将采样后的特征作为检测特征传入异常评分器，完成评分器的初始化
self.anomaly_scorer.fit(detection_features=[features])
```

代码 2-4 构建记忆库

最后是构建记忆库的代码实现，其中，`featuresampler` 表示核心集子采样，在 2.4 中详细介绍。

2.4 核心集采样

2.4.1 内存库规模过大

随着训练集规模增大， M 会变得极其庞大，导致新数据所需的推理时间以及存储需求也会增加。SPADE 中已经注意到了这个问题，由于计算限制，SPADE 需要一个较弱的特征提取的预训练模型，并对最后一个特征图进行全局平均。这会导致分辨率低、且偏向 ImageNet，降低异常检测性能。

要解决这个问题，需要让 M 保留空间分布特性，支持更精准局部匹配，以适应更大的图像，以及降低冗余，且缩减后的 M 仍要覆盖所有正常特征的分布范围，根据文献[6]的结果，采用随机丢弃特征的方法，以及采用较大的步幅 s 来进行采样的方法，导致性能大幅降低，会损失 M 中正常特征范围里的大量信息，PatchCore 选择一种核心集子采样的方法，来保证性能同时降低推理时间。

2.4.2 选择核心集

核心集是从原始集合中筛选的小规模子集，可最大限度保留原始集合的关键结构与分布。对于不同的问题，核心集也会不同，由于 PatchCore 会通过计算最近邻来确定异常分数，因此，选择极小极大设施定位[7]的方法来选择核心集。以确保 M 的核心集 M_c 在 patch 特征空间中覆盖的范围与原始记忆库 M 覆盖的范围大致相似。

极小极大设施定位是设施选址问题（Facility Location Problem）的一个变体，起源于运筹学与组合优化领域，这个问题的目标是，在给定空间中选择有限个设施的位置，使得所有用户到最近设施的最大距离最小化。极小极大的含义就是要最小化最大距离，优先改善距离设施最远的用户的距离。

如果把记忆库中所有 patch 特征看作设施，那么就是要选择一部分 patch 特征，使得所有特征到这一部分特征的最大距离最小化，即公式 2-3 所示， $|m - n|_2$ 为原始特征 m 与核心集特征 n 的欧氏距离， $\min_{n \in M_c} |m - n|_2$ 表示 m 到核心集的最

近距离， $\max_{m \in M}(\cdot)$ 即取这些最近距离的最大值。

$$\mathcal{M}_c^* = \arg \min_{\mathcal{M}_c \subset \mathcal{M}} \max_{m \in \mathcal{M}} \min_{n \in \mathcal{M}_c} |m - n|_2 \quad (2-3)$$

要精确计算核心集是 NP 难的[8]，为了近似求解，使用文献[9]提出的贪心算法，为进一步减少计算核心集的时间，使用随机线性投影[10]的方法，将高维特征降至低维空间，在低维空间中计算距离以替代高维空间距离。贪心算法的步骤如下：

- (1) 初始化：核心集 M_C 为空集，设定核心集的规模 l
- (2) 迭代 l 次，对每次迭代计算原始内存库中所有未入选特征 $m \in \mathcal{M} - M_C$ 到当前 M_C 的最近距离，选择最近距离最大的特征 m_i 加入 M_C
- (3) 迭代结束后，输出 M_C 。

使用 PatchCore- $n\%$ 来表示原始记忆库被子采样到的比 n ，例如，PatchCore-1% 表示 M 被缩减为原来的 1/100。

该算法的效果由图 2-2 所示，图中蓝色点代表的完整数据集，包含数据的真实分布特征，红色点代表从原始数据中抽取的子集，用于近似原始数据的分布，随机采样算法是完全随机抽取子集，不考虑原始数据的分布特征。

根据图 2-2a，多模态分布指原始数据天然形成多个离散的聚类簇，使用核心集采样的方法均匀覆盖了所有蓝色聚类簇，每个簇都有代表性样本被保留，使用随机采样的方法导致红色点集中在部分聚类簇中间部分，这些簇的外周并不能很好的覆盖。

根据图 2-2b，均匀分布指原始数据在特征空间中呈均匀密集分布，使用核心集采样的方法得到的红色点在特征空间中呈现均匀排布，与原始蓝色点的密度一致，没有明显的密集或稀疏区域，保证了子集能近似原始数据的全局覆盖范围。使用随机采样得到的红色点出现局部聚集或缺，破坏了原始数据的均匀性。

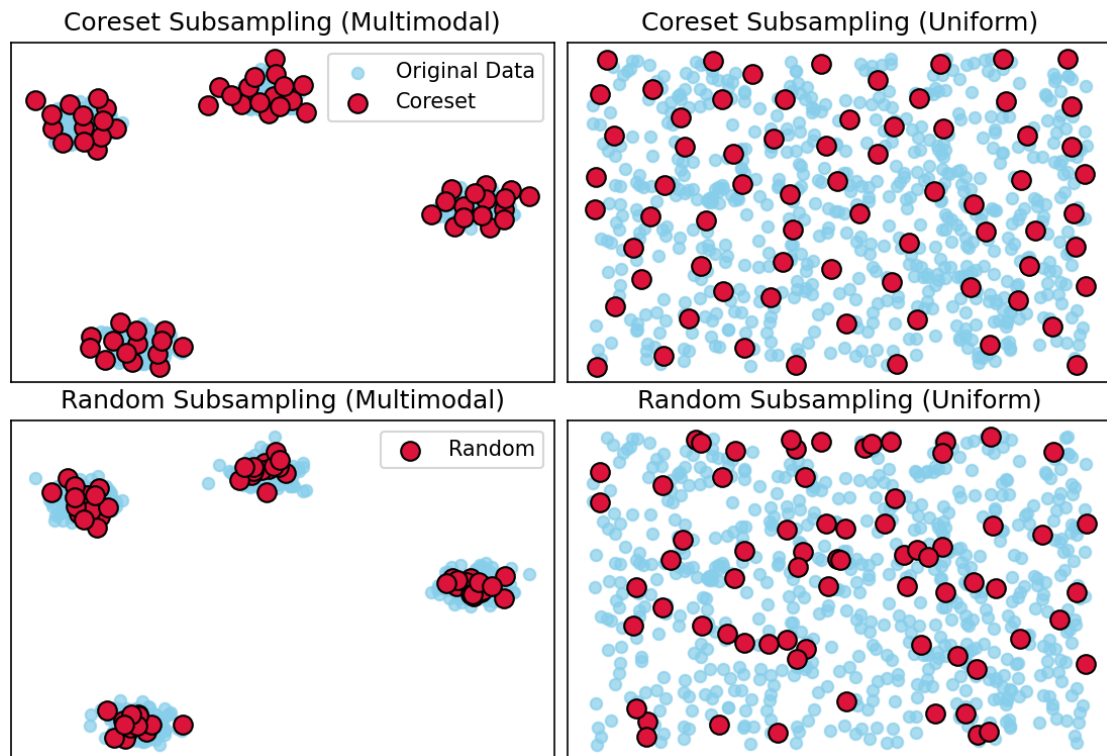


图 2-2 不同采样方法对比
图 a: 使用核心集采样方法（上）
图 b: 使用随机采样（下）

2.4.4 代码实现

```

class GreedyCoresetSampler(BaseSampler): # 基于贪心算法的核心集采样器
    def __init__(
        self,
        percentage: float, # 采样比例
        device: torch.device, # 计算设备 (GPU/CPU)
        dimension_to_project_features_to=128, # 特征降维后的目标维度, 默认 128
    ):
        """贪心核心集采样的基类"""
        super().__init__(percentage) # 调用父类构造函数, 传入采样比例

        self.device = device # 保存计算设备
        # 保存特征降维的目标维度
        self.dimension_to_project_features_to = dimension_to_project_features_to

    def _reduce_features(self, features):
        """对特征进行降维处理, 降低后续距离计算复杂度"""
        # 若特征已满足目标维度, 直接返回无需降维
        if features.shape[1] == self.dimension_to_project_features_to:
            return features
        # 定义线性投影层 (无偏置), 实现特征维度转换
        mapper = torch.nn.Linear(
            features.shape[1], self.dimension_to_project_features_to, bias=False
        )
        mapper.to(self.device) # 将投影层迁移到指定设备
        features = features.to(self.device) # 将特征迁移到指定设备
        return mapper(features) # 执行降维并返回结果

    def run(
        self, features: Union[torch.Tensor, np.ndarray]
    ) -> Union[torch.Tensor, np.ndarray]:
        """
        执行贪心核心集采样
        Args:
            features: 输入特征集合, shape 为 [N x D] (N=特征总数, D=特征维度)
        Returns:
            采样后的特征集合, 保持与输入相同的数据类型
        """
        # 若采样比例为 1, 直接返回原始特征
        if self.percentage == 1:
            return features

```

```

self._store_type(features) # 记录输入特征的数据类型 (numpy/torch)
# 若输入为 numpy 数组, 转换为 torch 张量便于计算
if isinstance(features, np.ndarray):
    features = torch.from_numpy(features)
# 对特征进行降维优化
reduced_features = self._reduce_features(features)
# 计算贪心核心集的采样索引
sample_indices = self._compute_greedy_coreset_indices(reduced_features)
# 根据索引筛选特征, 并恢复为原始数据类型
features = features[sample_indices]
return self._restore_type(features)

@staticmethod
def _compute_batchwise_differences(
    matrix_a: torch.Tensor, matrix_b: torch.Tensor
) -> torch.Tensor:
    """
    基于 PyTorch 批量计算欧氏距离
    Args:
        matrix_a: 特征矩阵 A, shape 为[N x D]
        matrix_b: 特征矩阵 B, shape 为[M x D]
    Returns:
        距离矩阵, shape 为[N x M], 元素(i,j)表示 A[i]与 B[j]的欧氏距离
    """
    # 计算 matrix_a 中每个向量的 L2 范数平方, shape 转为[N, 1]
    a_times_a = matrix_a.unsqueeze(1).bmm(matrix_a.unsqueeze(2)).reshape(-1, 1)
    # 计算 matrix_b 中每个向量的 L2 范数平方, shape 转为[1, M]
    b_times_b = matrix_b.unsqueeze(1).bmm(matrix_b.unsqueeze(2)).reshape(1, -1)
    # 计算 matrix_a 与 matrix_b 的点积, shape 为[N, M]
    a_times_b = matrix_a.mm(matrix_b.T)

    # 欧氏距离公式:  $\sqrt{||a||^2 + ||b||^2 - 2ab}$ , clamp(0, None)确保非负 (数值稳定性)
    return (-2 * a_times_b + a_times_a + b_times_b).clamp(0, None).sqrt()

def _compute_greedy_coreset_indices(self, features: torch.Tensor) -> np.ndarray:
    """
    执行迭代式贪心核心集选择, 获取采样索引
    Args:
        features: 输入特征库, shape 为[NxD]
    Returns:
        采样索引数组, shape 为[K] (K=采样后特征数)
    """
    # 计算全量特征的距离矩阵 (N x N)
    distance_matrix = self._compute_batchwise_differences(features, features)

```

```

# 初始化锚点距离：每个特征到所有特征的 L2 范数（衡量特征“孤立程度”）
coreset_anchor_distances = torch.norm(distance_matrix, dim=1)

coreset_indices = [] # 存储核心集采样索引
# 计算需要采样的核心集大小（总特征数×采样比例）
num_coreset_samples = int(len(features) * self.percentage)

# 迭代选择核心集特征
for _ in range(num_coreset_samples):
    # 选择锚点距离最大的特征（最孤立，最具代表性）
    select_idx = torch.argmax(coreset_anchor_distances).item()
    coreset_indices.append(select_idx)

    # 取出所有特征到当前选中特征的距离（N x 1）
    coreset_select_distance = distance_matrix[
        :, select_idx : select_idx + 1 # noqa E203
    ]
    # 合并原锚点距离与新选中特征的距离
    coreset_anchor_distances = torch.cat(
        [coreset_anchor_distances.unsqueeze(-1), coreset_select_distance], dim=1
    )
    # 更新锚点距离：取每个特征到已选核心集的最小距离
    coreset_anchor_distances = torch.min(coreset_anchor_distances, dim=1).values

return np.array(coreset_indices)

```

代码 2-5 核心集采样实现

2.5 实现异常检测

基于前文构建的 patch 特征内存库，在本节中，要介绍如何实现异常检测，即计算异常分数。

2.5.1 计算图像级异常评分

对测试图像 x^{test} ，执行与训练阶段一致的特征处理流程，包括用 ImageNet 预训练模型提取中层特征图，通过局部邻域聚合与步幅采样，生成 patch 特征集合 $\mathcal{P}(x^{\text{test}}) = \mathcal{P}_{s,p}(\phi_j(x^{\text{test}}))$ ，每个 patch 特征对应原始测试集图像的一个局部区域。

我们计算图像级异常得分基于一个假设，即单个 patch 异常即可判定图像异常，通过近邻搜索找候选异常 patch，并计算基础分数 s^* ，然后对分数进行加权修正，得到最终的评分。

2.5.1.1 计算基础异常评分

对 $\mathcal{P}(x^{\text{test}})$ 的每个测试 patch 特征 m^{test} ，计算其与核心集 M_c 的所有正常特

征 m 的欧氏距离，找到每个 m^{test} 的最近邻特征 m^* 及对应的最小距离，然后选取所有最小距离中的最大值作为基础分数 s^* ，对应的测试 patch $m^{test,*}$ 即为图像中与正常模式差异最大的疑似异常区域，对应的公式为 (2-4)、(2-5) 所示。

$$m^{test,*}, m^* = \arg \max_{m^{test} \in \mathcal{P}(x^{test})} \arg \min_{m \in \mathcal{M}_c} |m^{test} - m|_2 \quad (2-4)$$

$$s^* = |m^{test,*} - m^*|_2 \quad (2-5)$$

2.5.1.2 重加权

基础分数 s^* 可能受到干扰，例如，记忆库中与异常候选 $m^{test,*}$ 距离最远的 m^* 本身就是距离其他正常的 patch 距离较远，从而导致，即使 s^* 这个距离看上去较大，但实际上，依然不算具有异常。为了避免这一情况，找到核心集中与 m^* 距离最近的 b 个特征，组成邻域集合 $\mathcal{N}_b(m^*)$ ，它反映了 m^* 在正常特征空间中的密集程度，然后通过 Softmax 函数计算 $m^{test,*}$ 与 $\mathcal{N}_b(m^*)$ 中所有特征的距离占比，如果 m^* 本身就是比较孤立的，那么乘上较小的系数，否则，保留，数学公式如 (2-6) 所示。

$$s = \left(1 - \frac{\exp ||m^{test,*} - m^*||_2}{\sum_{m \in \mathcal{N}_b(m^*)} \exp ||m^{test,*} - m||_2}\right) \cdot s^* \quad (2-6)$$

2.5.2 代码实现

```
def predict(self, data):
    """异常检测推理入口：根据输入数据类型分发推理任务"""
    # 若输入为 DataLoader（批量数据集），调用批量推理方法
    if isinstance(data, torch.utils.data.DataLoader):
        return self._predict_dataloader(data)
    # 若输入为单批次图像，直接调用单批次推理方法
    return self._predict(data)

def _predict_dataloader(self, dataloader):
    """为整个 DataLoader 提供异常分数和异常掩码（大规模数据推理）"""
    # 将前向传播模块设为评估模式（推理阶段不训练）
    _ = self.forward_modules.eval()

    scores = [] # 存储所有图像的图像级异常分数
    masks = [] # 存储所有图像的像素级异常掩码
    labels_gt = [] # 存储数据集的真实异常标签（1=异常，0=正常）
    masks_gt = [] # 存储数据集的真实异常掩码（像素级标注）
    # 使用 tqdm 创建推理进度条，desc 为进度条描述，leave=False 不保留完成后的进度条
    with tqdm.tqdm(dataloader, desc="Inferring...", leave=False) as data_iterator:
        for image in data_iterator:
            # 若输入是字典（含图像、标签、掩码等结构化数据）
            if isinstance(image, dict):
                # 提取真实异常标签并转为列表添加到 labels_gt
```

```

        labels_gt.extend(image["is_anomaly"].numpy().tolist())
        # 提取真实异常掩码并转为列表添加到 masks_gt
        masks_gt.extend(image["mask"].numpy().tolist())
        # 仅保留图像数据用于推理
        image = image["image"]
        # 对当前批次图像执行推理，得到异常分数和掩码
        _scores, _masks = self._predict(image)
        # 逐样本将结果添加到总列表
        for score, mask in zip(_scores, _masks):
            scores.append(score)
            masks.append(mask)
        # 返回推理结果：预测分数、预测掩码、真实标签、真实掩码
        return scores, masks, labels_gt, masks_gt

def _predict(self, images):
    """对单批次图像推理，输出图像级异常分数和像素级异常掩码"""
    # 将图像转为浮点型并迁移到指定计算设备（GPU/CPU）
    images = images.to(torch.float).to(self.device)
    # 前向传播模块设为评估模式
    _ = self.forward_modules.eval()

    batchsize = images.shape[0] # 获取当前批次的图像数量
    # 关闭梯度计算，减少内存占用并提升推理速度
    with torch.no_grad():
        # 提取图像的补丁特征和补丁空间尺寸（调用_embed方法，provide_patch_shapes=True 返回尺寸
        # 信息）
        features, patch_shapes = self._embed(images, provide_patch_shapes=True)
        features = np.asarray(features) # 转为 numpy 数组，适配后续评分器

        # 调用异常评分器计算补丁级异常分数（初始与图像级分数共用变量，后续分别处理）
        patch_scores = image_scores = self.anomaly_scorer.predict([features])[0]

        # ----- 图像级异常分数计算 -----
        # 将扁平的补丁分数恢复为批次结构（[批次×补丁数]）
        image_scores = self.patch_maker.unpatch_scores(
            image_scores, batchsize=batchsize
        )
        # 展平特征维度，统一为[批次×补丁数×特征维度]格式
        image_scores = image_scores.reshape(*image_scores.shape[:2], -1)
        # 聚合补丁分数为图像级分数（递归取最大值，符合"单补丁异常即图像异常"逻辑）
        image_scores = self.patch_maker.score(image_scores)

        # ----- 像素级异常掩码生成 -----
        # 将扁平的补丁分数恢复为批次结构

```

```

patch_scores = self.patch_maker.unpatch_scores(
    patch_scores, batchsize=batchsize
)
scales = patch_shapes[0] # 取第一个层级的补丁空间尺寸（如[56,56]）
# 将补丁分数重塑为[批次×补丁高×补丁宽]，与特征图的补丁分布对齐
patch_scores = patch_scores.reshape(batchsize, scales[0], scales[1])
# 将补丁级分数转换为像素级异常掩码（通过插值匹配原始图像分辨率）
masks = self.anomaly_segmentor.convert_to_segmentation(patch_scores)

# 以列表形式返回逐样本的图像级分数和异常掩码
return [score for score in image_scores], [mask for mask in masks]

class NearestNeighbourScorer(object):
    """基于最近邻（k-NN）的异常评分器，核心用于计算补丁与记忆库的距离"""
    def __init__(self, n_nearest_neighbours: int, nn_method=FaissNN(False, 4)) -> None:
        """
        初始化最近邻异常评分器
        Args:
            n_nearest_neighbours: 用于计算异常分数的近邻数量（如 5 个近邻）
            nn_method: 近邻搜索方法（默认 FaissNN，高效大规模特征检索工具）
        """
        self.feature_merger = ConcatMerger() # 特征合并器（用于聚合多层次特征）

        self.n_nearest_neighbours = n_nearest_neighbours # 近邻数量
        self.nn_method = nn_method # 近邻搜索实现

        # 图像级近邻搜索匿名函数：查询每个补丁的 n 个近邻
        self.imagelevel_nn = lambda query: self.nn_method.run(
            n_nearest_neighbours, query
        )
        # 像素级近邻搜索匿名函数（预留，用于更精细的像素级检索）
        self.pixelwise_nn = lambda query, index: self.nn_method.run(1, query, index)

    def fit(self, detection_features: List[np.ndarray]) -> None:
        """
        构建记忆库的近邻搜索索引
        Args:
            detection_features: 训练阶段的特征列表，格式为[[批次×特征维度]...]
            包含骨干网络输出的、用于图像级异常检测的特征向量（或特征图）
        """
        # 合并多层次特征（如 ResNet 的 2 层和 3 层特征）
        self.detection_features = self.feature_merger.merge(
            detection_features,

```

```

    )
    # 用合并后的训练特征（记忆库）训练近邻搜索方法，构建检索索引
    self.nn_method.fit(self.detection_features)

def predict(
    self, query_features: List[np.ndarray]
) -> Union[np.ndarray, np.ndarray, np.ndarray]:
    """
    预测异常分数：在训练记忆库中搜索测试特征的近邻并计算距离
    Args:
        query_features: 测试图像的特征列表，格式与训练特征一致
    Returns:
        anomaly_scores: 补丁级异常分数（每个补丁的平均近邻距离）
        query_distances: 每个补丁到 n 个近邻的距离矩阵
        query_nns: 每个补丁的 n 个近邻在记忆库中的索引
    """
    # 合并测试图像的多层级特征
    query_features = self.feature_merger.merge(
        query_features,
    )
    # 执行近邻搜索，得到每个测试补丁的 n 个近邻距离和索引
    query_distances, query_nns = self.imagelevel_nn(query_features)
    # 计算异常分数：取每个补丁到 n 个近邻的平均距离
    anomaly_scores = np.mean(query_distances, axis=-1)
    return anomaly_scores, query_distances, query_nns

@staticmethod
def _detection_file(folder, prepend=""):
    """返回特征库的保存路径"""
    return os.path.join(folder, prepend + "nnscorer_features.pkl")

@staticmethod
def _index_file(folder, prepend=""):
    """返回近邻搜索索引的保存路径"""
    return os.path.join(folder, prepend + "nnscorer_search_index.faiss")

@staticmethod
def _save(filename, features):
    """将特征数据保存为 pkl 文件"""
    if features is None:
        return
    with open(filename, "wb") as save_file:
        # 使用最高协议保存 pickle 文件，提升兼容性和压缩率
        pickle.dump(features, save_file, pickle.HIGHEST_PROTOCOL)

```

```

@staticmethod
def _load(filename: str):
    """从 pkl 文件加载特征数据"""
    with open(filename, "rb") as load_file:
        return pickle.load(load_file)

def save(
    self,
    save_folder: str,
    save_features_separately: bool = False,
    prepend: str = "",
) -> None:
    """
    保存近邻搜索索引和特征库
    Args:
        save_folder: 保存文件夹路径
        save_features_separately: 是否单独保存特征库（默认不保存）
        prepend: 文件名前缀（用于区分不同版本的保存文件）
    """
    # 保存近邻搜索索引
    self.nn_method.save(self._index_file(save_folder, prepend))
    # 若需要单独保存特征库，调用_save方法保存
    if save_features_separately:
        self._save(
            self._detection_file(save_folder, prepend), self.detection_features
        )

def save_and_reset(self, save_folder: str) -> None:
    """保存数据后重置近邻索引（释放内存）"""
    self.save(save_folder)
    self.nn_method.reset_index()

def load(self, load_folder: str, prepend: str = "") -> None:
    """
    加载保存的近邻搜索索引和特征库
    Args:
        load_folder: 加载文件夹路径
        prepend: 文件名前缀（与保存时一致）
    """
    # 加载近邻搜索索引
    self.nn_method.load(self._index_file(load_folder, prepend))
    # 若特征库文件存在，加载特征库
    if os.path.exists(self._detection_file(load_folder, prepend)):

```

```

self.detection_features = self._load(
    self._detection_file(load_folder, prepend)
)

```

代码 2-6 计算图像级异常评分

2.6 总结

最后，概述 PatchCore 方法的全部流程，首先，在训练阶段，正常样本通过预训练特征提取器，然后被分解为具有邻域聚合的 patch 特征的存储库。为了降低冗余并减少推理时间，通过贪心核心集采样对该存储库进行下采样。在测试阶段，如果至少有一个 patch 是异常的，则图像被分类为异常，如图 2-3 所示。

和一般的模型在训练过程中需要多次遍历数据集，根据定义的损失函数计算预测值与真实值之间的误差，并利用梯度下降等优化算法不断调整模型参数，以最小化损失函数不同，PatchCore 不需要训练 epoch，它利用预训练模型直接提取图像特征，构建记忆库，并且在进行核心集采样与计算异常分数时，也不需要依赖损失函数，因为这种方式不涉及模型参数的调整，而是依赖于特征匹配和距离计算。

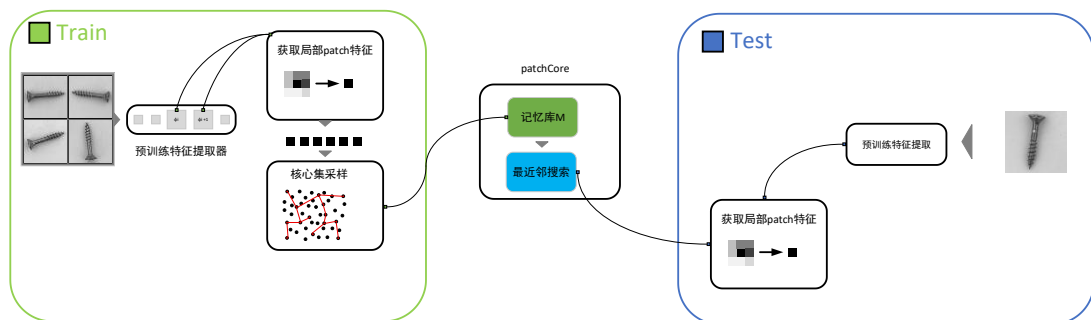


图 2-3 PatchCore 算法流程

虽然本任务是进行图像级别的异常检测，但是在工业场景下，更加重要的是进行像素级的异常检测，即确定具体异常的位置，关于实现这一目标的方法，我们在第三章中进行说明。

第三章 实验与扩展

3.1 模型参数分析

PatchCore 模型的一些超参数为：使用的预训练特征提取器以及所提取特征图的层级、下采样的百分比、patch 邻域大小。

下面分别具体介绍一下这些超参数：

对于预训练特征提取器，在 `backbones.py` 中定义的包括 WideResNet50，特征丰富度高，泛化性强，这是默认的选项，因为它是平衡了性能与速度；WideResNet101，可以处理更高的分辨率，特征表达更强；ResNet50/101，是较轻量化的特征提取器，速度快。

特征图提取层级是从特征提取器中选择的中间层，用于生成特征图，为后续的 patch 特征做准备，具体来说，层级共有 4 个，layer1 属于浅层，layer2,3 属于中层，layer4 属于深层。

下采样百分比是使用核心集采样的方法对 patch 记忆库进行降采样的比例，在保留分布覆盖度的同时减少冗余。

Patch 邻域大小：这是生成 patch 特征的时候的邻域窗口大小，可以控制每个 patch 的上下文信息，1x1 为小邻域，不包含上下文信息，3x3 为中等大小邻域，5x5，7x7 为大邻域。

进行测试的平均图像级 AUROC 指标含义是对 5 个类别进行训练和评估，得到的 5 个类别的图像级 AUROC 求其平均值。

3.1.1 预训练模型的影响

在本节中，我们使用不同的预训练模型来进行特征提取，以此来探究该方面的影响，结果由表 3-1 所示，所有测试的骨干网络均实现了 98.6% 以上的图像级 AUROC，表明 PatchCore 对基准模型的兼容性强，不存在因骨干选择导致的性能骤降问题。WideResNet101 的宽卷积设计能提取更丰富的中高频特征，与 PatchCore 依赖的局部 patch 高度适配，因此在较高采样比例下性能最大化。WideResNet50、WideResNet101 这两种网络在不同采样比例下的波动较小，鲁棒性强，适合对采样比例不敏感的工业场景；ResNet50 波动较高，在实际应用中要使用较高的采样比例才能保证性能。

模型	%n	平均图像级 AUROC
ResNet50	10	99.0
ResNet50	1	98.7
WideResNet50	10	98.9
WideResNet50	1	99.0
ResNet101	10	98.6
ResNet101	1	98.7
WideResNet101	10	99.1
WideResNet101	1	99.0
ResNeXt101	10	98.9
ResNeXt101	1	98.7

表 3-1 不同模型的性能

3.1.2 提取特征图层级的影响

在本节中，使用不同的层级及其层级的组合，进行训练和评估，以探究其影响，结果如图 3-1 所示，单层级特征的性能呈现先升后降的规律，层级 2 和层级 3 表现优于层级 1 和层级 4，因为层级 1 对应网络浅层输出，抽象度极低，受噪声影响大，导致模型难以捕捉有意义的异常信号；层级 2 和 3 属于网络中间层输出，平衡了局部空间信息与语义抽象，保留了较高的空间分辨率，能捕捉工业缺陷的位置且具备一定语义抽象能力；层级 4 对应网络深层输出，抽象度最

高但分辨率最低，ImageNet 偏见严重且丢失空间信息。

多层次特征融合的性能普遍优于单一级别，其中 2+3 组合达到峰值，而包含层级 1 或层级 4 的组合性能反而受限，因为层级 2 和 3 同属中层级，二者融合可实现互补，而层级 1 会引入噪声，干扰中层级特征，层级 4 的特征虽然具有宏观语义，但受 ImageNet 偏见和空间分辨率低的限制，无法带来增益。因此，使用层级 2, 3 的融合特征是最优的方案。

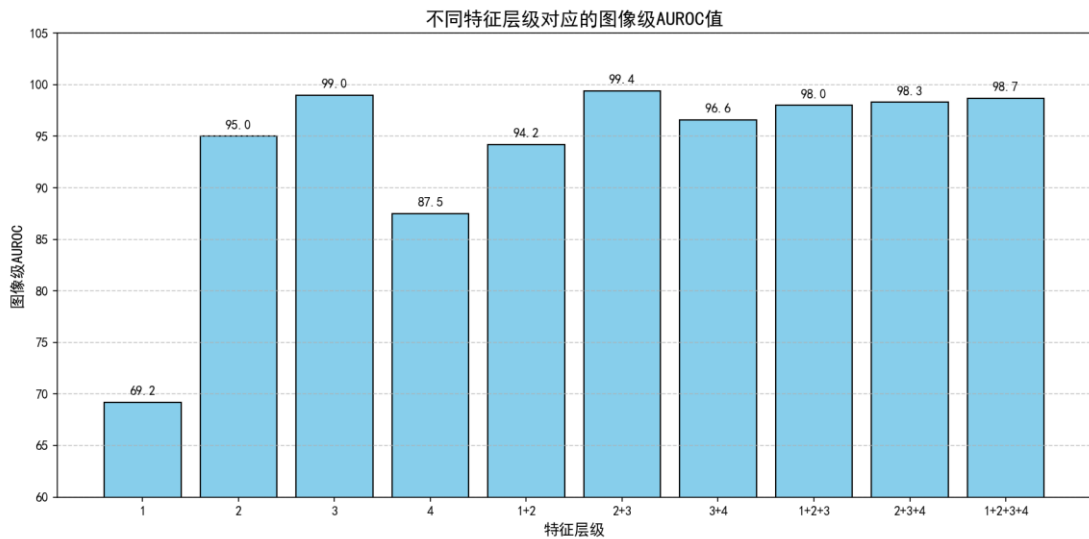
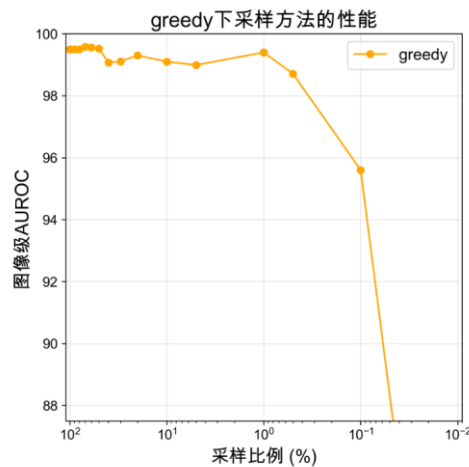


图 3-1 不同层级模型性能的区别

3.1.3 下采样百分比

在本节中，探索不同的核心集采样的百分比对模型性能的影响，结果由图 3-2 所示，Greedy Coreset 采样在极宽的采样比例范围内均能保持接近最优的图像级 AUROC 性能，只有采样比例极低时性能才显著下降。



至 70% 采样比例时达到 99.58% 的峰值，略高于全量采样。因为全量采样存在大量冗余特征，核心集采样通过剔除冗余样本，反而提升了特征匹配的效率，在这个区间内，性能饱和，冗余度高，因此无明显衰减。在中低采样比例时（1-20），AUROC 最低为 98.71%，最高达 99.4%，整体维持在 98.7-99.4 的水准，性能微降但仍保持极高水平，因为核心集采样的优势是最小化最大距离误差，即使采样比例很低，仍能通过精选样本保留特征的整体分布结构。极低采样比例时，AUROC 骤降，性能断崖式下降，因为采样比例过低时，无法覆盖数据的特征空间，导致测试时异常 patch 无法找到匹配的特征，因此，虽然核心集采样更加的抗衰减，但是仍然受限于样本数量不足，难以支撑近似的数据分布。

更进一步，我们继续探索该方法的有效性。图 3-3 将核心集采样方法与随机采样方法进行对比，发现核心集采样在几乎所有采样比例下均显著优于随机采样，且采样比例越低，两者性能差距越大。

高采样比例下，随机采样能够保留大部分特征，甚至存在不少冗余，特征空间覆盖度尚可，因此与核心集采样的性能差异较小。但核心集采样通过剔除无效冗余，在较多比例情况下，性能略微高于随机采样方法。在中低采样下，随机采样方法信息丢失加速，即随机选取的样本难以覆盖数据的特征分布，而核心集采样更能够精准筛选重要的特征，因此性能几乎无衰减。

通过以上论据表明，核心集采样方法是相当有效的。

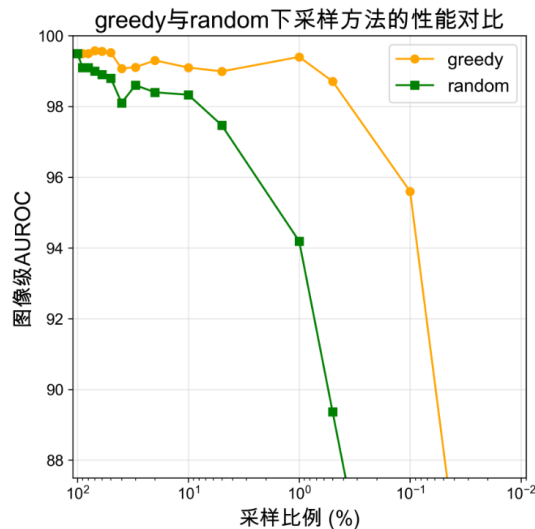


图 3-3 核心集采样 vs 随机采样

3.1.4 Patch 邻域大小

在本节，我们将探索 Patch 邻域大小对模型性能的影响，实验结果由图 3-4 所示，PatchCore 的核心之一就是使用邻域聚合的 Patch 特征构建记忆库，Patch 邻域大小会直接影响特征的有效性，Patch 大小需要满足两部分：

保留局部信息：工业异常常表现为细微缺陷，需要高分辨率特征；

捕捉足够空间上下文：单一像素缺乏周围环境信息，易受噪声干扰，需要通过邻域聚合引入局部关联性。

当补丁大小为 3 时，恰好实现了两者的平衡，能够覆盖缺陷周围的正常纹理上下文，避免孤立；也没有过度扩大范围导致抽象度过高，印证了在方法一章中的分析。

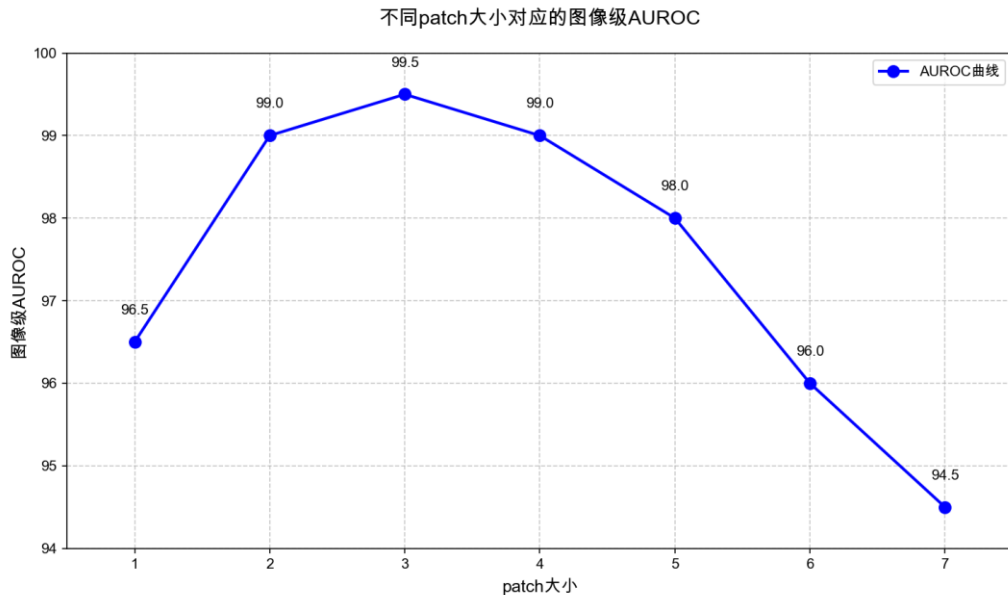


图 3-4 不同 Patch 大小影响

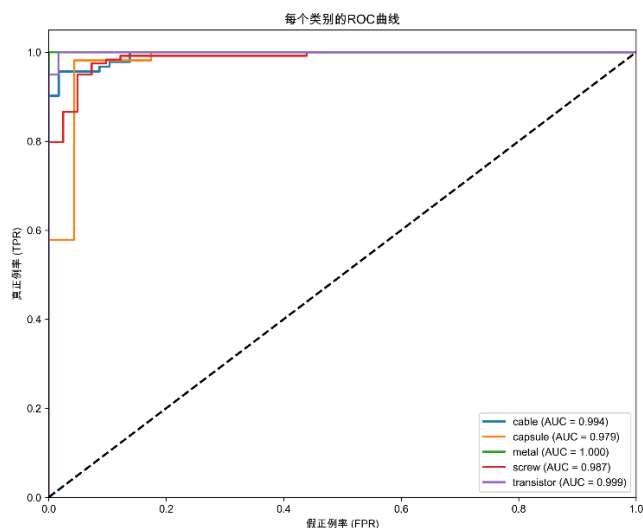
3.1.5 总结

在本节，我们探索了一些对于 PatchCore 性能影响较大的参数，并得出了一些结论：

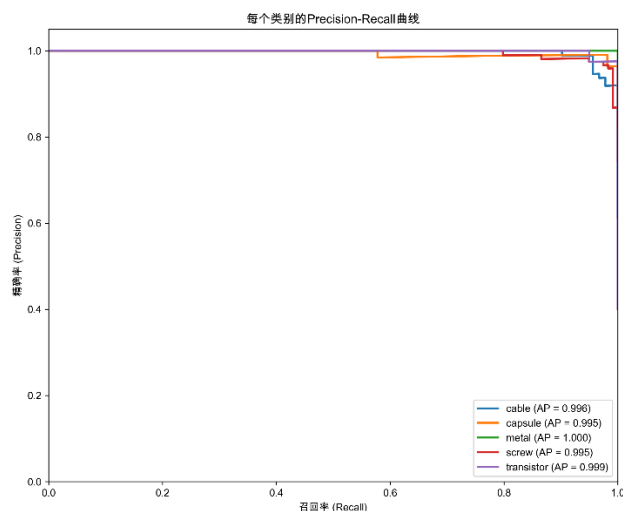
PatchCore 对于预训练特征提取器的兼容性强，各模型的 AUROC 均为优秀水平，WideResNet 系列鲁棒性强，其中 WideResNet101 特征表达强，适配性最佳。对于特征图提取层级，中层级（2、3）性能最优，二者融合达到最高，浅层易受噪声干扰，深层存在 ImageNet 偏见且分辨率低，均不适宜。对于下采样百分比，核心集采样在 1%-100% 比例内性能稳定，仅在比例极低时性能骤降，显著优于随机采样。对于 Patch 邻域大小， 3×3 最优，平衡了局部信息与空间上下文。这些探索一方面帮助我们得到最优的参数设置，另一方面，更重要的也是验证了该机制的有效性，例如，对于 patch 大小，当 p 为 1，即不存在上下文的时候，性能有较大降低，说明了该机制的有效。

我们还有一些没有进行探索的参数，包括预训练特征通道数、目标特征通道数、异常分数最近邻数量，因为经过实验，发现他们对于 PatchCore 的性能影响较为直观，例如，对于通道数，较高时，需要较大的计算开销，较低时，信息损失较大，实际应用与从预训练模型提取的特征图通道数相同即可；对于近邻数量，为 1 时更好的适配异常检测任务，其他数量具有较高的鲁棒性等，不具有进行较多讨论的价值。

图 3-5 展示了在最优的参数组合下，模型的性能可视化，包括 ROC 曲线以及召回曲线，根据可视化的结果，模型的图像级 AUROC 性能极其优秀，对于 1.2 节中的任务可以很好的完成。



(a) ROC 曲线



(b) 召回曲线

图 3-5 模型性能可视化

3.2 推理时间分析

在本节中，探究从另一方面判定模型性能指标，即推理时间，因为工业场景中，对检测流程的时间有严格约束，生产线的连续运转不允许模型出现过长延迟，推理时间关系到模型部署的硬件资源消耗与成本，工业场景中硬件预算通常有限，因此对计算效率有一定的要求。表 3-2 测定了在 RTX 4060GPU 上，不同采样率的推理时间和图像级 AUROC，随着采样比例从 100%到 1%，单张图像推理延迟从 215ms 降至 44.6ms，证明核心集采样确实可以减少特征冗余、降低近邻搜索计算量，同时保证模型的性能，当采样率为 1%时，这样较短的推理时间可以满足在大部分场景下的要求。

方法	图像级 AUROC	延迟(ms/张)
PatchCore-1%	99.85	44.6
PatchCore-10%	99.63	73.8
PatchCore-100%	99.83	215

表 3-2 推理时间测定

3.3 少量样本异常检测实验

在本节中,探索 PatchCore 模型在少量训练样本下的性能,进行这样的探索,是因为工业异常检测的一大问题,即冷启动问题,意思是在实际生产中,获取大量正常样本已存在一定成本,而异常样本更极度稀缺,因此,本任务是基于正常样本的无监督学习,来进行异常检测。但是获取正常样本也存在一定成本,一些算法在全量正常样本的条件下表现可能较优,但是在少量样本场景下性能会大幅下降,这是因为它们依赖大量样本学习正常模式的统计分布,因此,本实验可以探究评估算法的,样本效率,即算法用最少的正常样本,能达到与全量样本接近的检测性能。

表 3-3 是实验的结果,所有类别都表现出训练样本数越多,图像级 AUROC 越高的趋势,因此, PatchCore 虽然样本效率较优,但是也符合一个直觉,即更多正常样本可以提供更丰富的特征分布信息,降低异常检测误差。不同品类在同样样本数下的 AUROC 差异较大,因为 MVTecAD 的各个类别的产品结构、缺陷类型不同,检测难度存在天然的差异,具体可分为两类情况:

高性能品类 (metal_nut、transistor、cable): 在 50 个样本时 AUROC 均接近或达到 99%, 10 个样本时 AUROC 也超过 83%。这是因为这类品类的正常特征分布相对规整,缺陷类型很容易通过 Patch 特征捕捉。

低性能品类 (screw、capsule): 例如 screw 在 50 个样本时 AUROC 仅有 80.77, 1 个样本时低至 42.69 等,这是因为部分工业品类自身特征复杂度高,需要更多样本才能刻画特征分布。

从总体上看,多数品类在 10 个样本时 AUROC 已超过 80%,即使是 1 个样本,部分品类的 AUROC 仍能维持在 70% 以上,得益于 PatchCore 的核心集采样,该机制能从极少样本中提取最具代表性的 Patch 特征,最大程度覆盖现有特征空间,表明 PatchCore 在样本效率这方面是极具竞争力的算法。

类别	训练样本数	图像级 AUROC
cable	1	73.28
	10	89.24
	50	98.31
capsule	1	71.50
	10	88.11
	50	94.06
metal_nut	1	62.71
	10	91.79
	50	99.22

screw	1	42.69
	10	61.32
	50	80.77
transistor	1	50.25
	10	83.13
	50	99.29

表 3-3 少样本异常检测结果

3.4 提高总体 AUROC 的小技巧

将预测的结果提交到 Kaggle 平台上进行评测，得到的分数并不是简单的对 5 个类别的 AUROC 取平均，而是计算总体的 AUROC，因此，如果直接把预测结果提交，会发现分数并不理想，由表 3-4 第一行所示。

因为不同类别的正常 / 异常样本分布差异较大，导致模型输出的异常分数在不同类别间处于不同的量级，例如，A 类物品的异常分数可能集中在 $[0,10]$ 区间，而 B 类物品的异常分数可能集中在 $[100,200]$ 区间，当计算总体 AUROC 时，会将所有类别样本的异常分数放在一起比较，会导致高尺度类别的正常样本可能被误判为异常，低尺度类别的异常样本可能被误判为正常。

为了提高这一分数，可以分别对每个类别的异常分数进行 z 标准化，将每个类别的异常分数转换为标准正态分布，这能够消除不同类别间异常分数的尺度差异，使所有类别的异常分数具有可比性。进行提交后，可以看到分数明显上涨，但依然和本地测量分别各个类别的得分有差异，由表 3-4 第二行所示，而如果使用模型预测训练集得到的异常分数均值和方差进行标准化，再次提交，可以发现，与分别测试的 AUROC 相当，由表 3-4 第三行所示。

这是因为标准化时使用的统计量来源不同，影响了异常分数的可比性和分布一致性。

MVTec AD 数据集中，每个类别的训练集只包含正常样本，用训练集的均值和方差标准化，就是将每个测试样本的异常分数与该类别的纯正常分布进行比对，经过这一过程后，测试样本中的正常样本的异常分数会集中在 0 附近，而异常样本会更远，如果使用测试集的统计量进行标准化，会混入异常样本的信息，导致正常/异常的边界模糊，因此，训练集统计量代表了模型学到的正常分布，用它标准化就能最大程度区分正常/异常。

不标准化	72.13
测试集统计量标准化	96.204
训练集统计量标准化	97.847

表 3-4 不同标准化区别

3.5 像素级定位实现

在 2.5.1 中已经实现了图像级的异常评分计算，那么，对于像素级的异常定位，我们有一个直观且简洁的想法，就是对 $\mathcal{P}(x^{\text{test}})$ 中的每一个 patch 特征，计算其异常分数，复用在这一节中的 patch 级分数计算逻辑，包括近邻搜索与重加权，

得到覆盖全图的 **patch** 级异常分数图，此时每个 **patch** 分数对应原始图像的一个局部区域，就将这个分数分配到原图中对应的像素，然后使用双线性插值将低分辨率的 **patch** 分数图上采样至与原始测试图像一致的尺寸，对插值后的像素级分数图应用高斯滤波，使异常区域轮廓更加平滑，得到能够可视化的异常分割图，而图中异常分数明显高于其他地方的那片区域就是异常所在的位置，由图 3-x 所示。

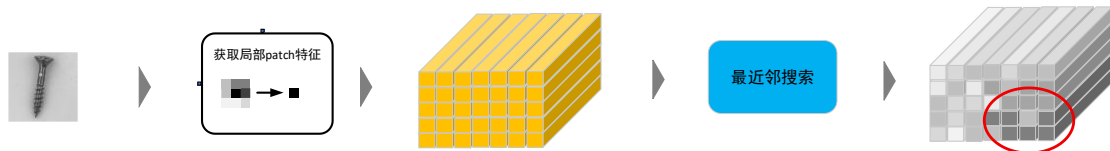
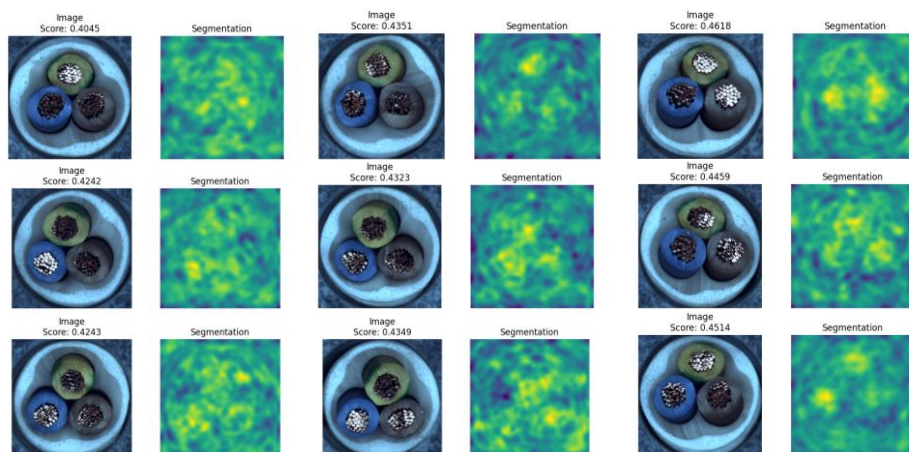


图 3-x 获取像素级异常分割图

而图像级异常分数，就是取上一步中，计算的所有 **patch** 异常分数中最大的那一个。

使用这个方法对 **cable** 进行预测的像素级结果如图 3-x 所示，其中，a 图为正常样本的异常分数与像素级预测结果，可知，对于正常样本的异常分数较低，训练集的平均异常分数为 0.42，对测试集进行预测的异常分数与训练集的异常分数相当，表示该模型可以较为有效的区分出正常样本，而对于正常样本的像素级结果，可知整张图片大部分 **patch** 的异常分数基本相同，均为较低水平，说明了在像素级层面，也可以准确的预测。b 图为测试集中异常样本的预测结果，图像级的异常分数均远大于正常样本，因此可以在图像级层面预测正确，而在像素级层面，可以有效的定位具体异常的位置，例如，b 图中展示的铜线异常、绝缘体撕裂，颜色错误，导线缺失这些类别，可以较为准确的标注出具体的位置。

(a)



(b)

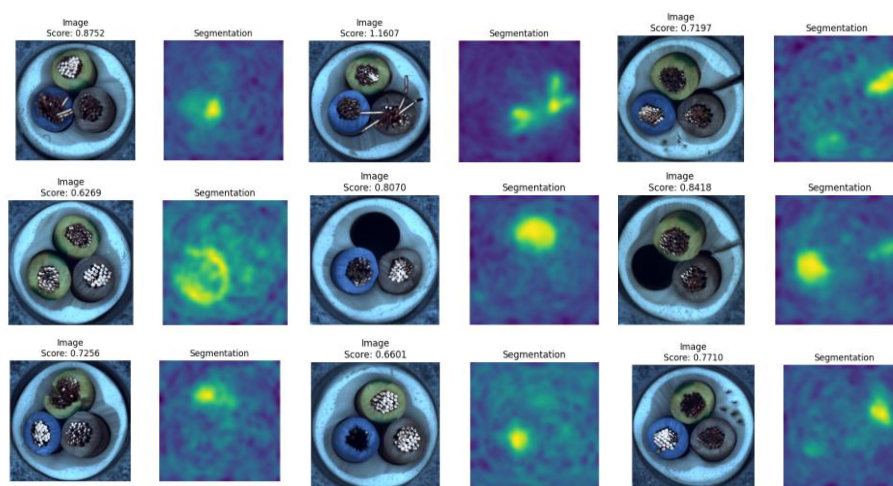


图 3-x cable 预测结果

图 3-x 显示了在其他 4 种类别上的预测结果，包括 1 张正常，2 张异常，结果表明，对于正常样本的异常分数较低，异常样本较高，均可以有效的预测，性能达到较高水平。

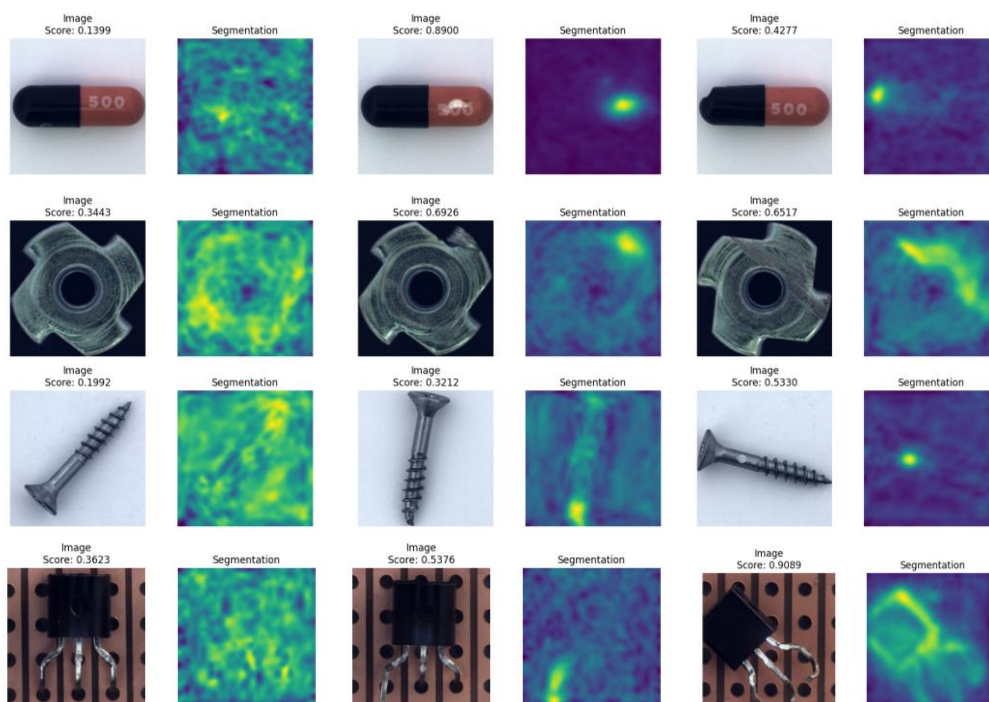


图 3-x 像素级预测结果

3.5.1 代码实现

```
def predict(
    self, query_features: List[np.ndarray]
) -> Union[np.ndarray, np.ndarray, np.ndarray]:
    """
    预测异常分数：在训练记忆库中搜索测试特征的近邻并计算距离
    Args:
        query_features: 测试图像的特征列表，格式与训练特征一致
    Returns:
        anomaly_scores: 补丁级异常分数（每个补丁的平均近邻距离）
        query_distances: 每个补丁到 n 个近邻的距离矩阵
        query_nns: 每个补丁的 n 个近邻在记忆库中的索引
    """
    # 合并测试图像的多层级特征
    query_features = self.feature_merger.merge(
        query_features,
    )
    # 执行近邻搜索，得到每个测试补丁的 n 个近邻距离和索引
    query_distances, query_nns = self.imagelevel_nn(query_features)
    # 计算异常分数：取每个补丁到 n 个近邻的平均距离
    anomaly_scores = np.mean(query_distances, axis=-1)
    return anomaly_scores, query_distances, query_nns
```

代码 3-1

首先, 代码 3-1 负责在训练阶段构建的记忆库中搜索测试样本的近邻, 通过距离计算输出所有 patch 的异常分数。

然后, 代码 3-2 负责将 patch 分数转换为像素级异常分割图, 通过 `unsqueeze(1)` 增加通道维度, 使用双线性插值将低分辨率的 patch 分数放大到原始图像尺寸。

最后, 代码 3-3 负责将原始图像、异常分数、像素级分割图组合成可视化结果并保存。

```
def convert_to_segmentation(self, patch_scores):

    with torch.no_grad():
        if isinstance(patch_scores, np.ndarray):
            patch_scores = torch.from_numpy(patch_scores)
            _scores = patch_scores.to(self.device)
            _scores = _scores.unsqueeze(1)
            _scores = F.interpolate(
                _scores, size=self.target_size, mode="bilinear", align_corners=False
            )
            _scores = _scores.squeeze(1)
            patch_scores = _scores.cpu().numpy()

    return [
        ndimage.gaussian_filter(patch_score, sigma=self.smoothing)
        for patch_score in patch_scores
    ]
```

代码 3-2

```
def plot_segmentation_images(
    savefolder,
    image_paths,
    segmentations,
    anomaly_scores=None,
    image_transform=lambda x: x,
    save_depth=1,
):
    """Generate anomaly segmentation images.

    Args:
        savefolder: 保存文件夹路径
        image_paths: List[str] 图像路径列表
        segmentations: [List[np.ndarray]] 生成的分割结果
        anomaly_scores: [List[float]] 每张图像的异常分数
        image_transform: [function or lambda] 可选的图像变换
        save_depth: [int] 用于图像保存名称的路径字符串数量
```

```

"""
if anomaly_scores is None:
    anomaly_scores = ["-1" for _ in range(len(image_paths))]
os.makedirs(savefolder, exist_ok=True)
for image_path, anomaly_score, segmentation in tqdm.tqdm(
    zip(image_paths, anomaly_scores, segmentations),
    total=len(image_paths),
    desc="Generating Segmentation Images...",
    leave=False,
):
    image = PIL.Image.open(image_path).convert("RGB")
    image = image_transform(image)
    if not isinstance(image, np.ndarray):
        image = image.numpy()
    min_val = image.min()
    max_val = image.max()
    if max_val > min_val: # 避免除以零
        image = (image - min_val) / (max_val - min_val)
    else:
        image = np.zeros_like(image) # 处理特殊情况
    # 使用正则表达式分割两种路径分隔符
    savename = re.split(r'[/\\]', image_path)
    # 过滤掉空字符串和可能的当前目录符号
    savename = [part for part in savename if part and part != '.']
    # 拼接最后 save_depth 个部分
    savename = "_".join(savename[-save_depth:])
    # 生成完整保存路径
    savename = os.path.join(savefolder, savename)
    f, axes = plt.subplots(1, 2)
    axes[0].imshow(image.transpose(1, 2, 0))
    axes[0].set_title(f"Image\nScore: {anomaly_score:.4f}")
    axes[0].axis('off')

    axes[1].imshow(segmentation)
    axes[1].set_title("Segmentation")
    axes[1].axis('off')

    f.set_size_inches(6, 3)
    f.tight_layout()
    f.savefig(savename)
    plt.close()

```

代码 3-3

第四章 开放性思考与讨论

虽然 PatchCore 方法在异常检测任务中具有相当优秀的性能，尤其是对本报告中的任务，图像级 AUROC 可以达到 0.99，基本可以认为能够满足任务，但是仍然存在一些可以改进的方面，在本节中，将会叙述这些方面，为后续的研究提供参考。

4.1 特征域偏移

PatchCore 方法依赖在 ImageNet 上预训练的 CNN 提取特征，并通过构建记忆库存储正常样本的特征嵌入。但这存在严重的域偏移问题，工业场景图像，比如 Mvtec ad 数据集的图像，与 ImageNet 数据集的场景差异很大，而该方法直接使用了预训练的特征提取器，且未经适配，那么提取的特征可能无法有效捕捉工业产品的细微缺陷，反而受自然场景特征的干扰，最终影响异常检测的精度。

另一方面，PatchCore 方法计算异常分数时，采用近邻搜索的方法，如果预训练特征本身与目标域不匹配，那么会进一步放大这种偏移，即使找到最近邻正常特征，距离计算的结果也无法真实的反映是否异常。

对于这种问题，可以设计一个特征适配器模块，用于将预训练特征投影到目标域适配的特征空间，使用线性变换修正 ImageNet 预训练特征的分布偏差，使特征更聚焦于工业产品的细节。文献[11]对这种问题进行改进，使用无偏置的单全连接层作为特征适配模块，因为它足够轻量化，以及使用线性变换，不会引入过度复杂的非线性映射，并进行了详细的实验来说明这种问题。如图 4-1 所示，绿色曲线不使用特征适配器，结果 Loss 最大，蓝色曲线表示使用单层全连接层，图片的结果表明这可以有效的减少 Loss，而黄色线是使用复杂的非线性特征适配器，该方法增加了计算复杂度，而且可能导致过拟合。

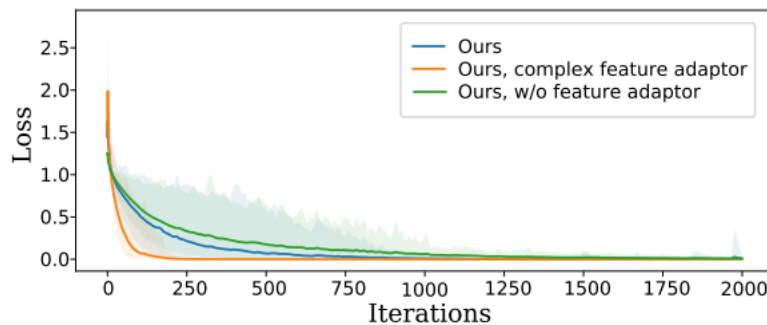


图 4-1 验证特征适配器有效性

4.2 多尺度异常捕捉能力不足

PatchCore 方法所建立的记忆库是只有单一感受野的 patch 特征，然后通过 patch 特征的距离计算来得到异常分数，在实现像素级异常检测的时候，就是使用所有的 patch 异常分数来生成补码图片，得到像素级的异常标注。在这个过程中，这种 patch 级的异常分数只有单一尺度，而现实是，工件有可能存在微小的

划痕、缺口、磨损这些高分辨率的特征，也有可能存在大范围的结构异常，比如颜色错误，电缆的铜线缺失这些较低分辨率的大范围异常。仅仅采用高分辨率就可能遗漏这些结构异常。

为了改善这个问题，可以构建一个 RRS（重建残差选择）模块，具体来说，将预训练特征提取器的各层的重建残差进行整合。对于低分辨率的重建残差，通过上采样操作使其分辨率提升至与最高分辨率的重建残差一致，随后沿着通道维度将它们串联起来，从而得到全局重建残差。这一步统一了不同分辨率的残差信息，为后续处理提供基础。

然后采用两种不同的池化策略对全局重建残差进行显著性评估，全局最大池化（GMP）主要突出局部区域的最大值，擅长捕捉小区域的局部异常。全局平均池化（GAP）更关注整体的平均响应，侧重于选择具有大范围的异常。

之后分别从 GMP 和 GAP 得到的结果中选取最大的元素，以此来索引重建残差的位置，从而得到具有最高最大值和平均值的重建残差，然后将这些被选中的残差进行拼接，保留最具判别力的多尺度信息。

将选择后的重建残差输入到判别器中，并将判别器的输出映射到图像级分辨率，得到像素级的异常检测结果。文献[12]使用这种方法进行改进，并进行了详细的消融实验，结果由表 4-1 所示，RRS 模块可以显著的提升性能，证明其有效性。

实验	RRS	图像级 AUROC
1	无	94.46 / 95.67
2	有	99.39 / 99.09

表 4-1 消融实验结果

4.3 计算效率低

前文已经讨论了 PatchCore 的流程，通过预训练卷积神经网络提取图像特征，再基于 k 近邻算法在特征空间中判断异常。但是 kNN 搜索环节存在显著的计算效率瓶颈，在推理阶段，根据训练过程中构建的特征数据库，为测试图像的每个特征向量寻找最相似的邻居样本，这一过程的时间复杂度随数据库规模增长而线性增加。

kNN 搜索依赖于将训练阶段的特征数据库存储在 GPU 内存中以加速查询，这导致模型的内存显著增加，即使已经进行了核心集采样，更进一步，kNN 搜索属于串行查找过程，难以充分利用 GPU 的并行计算架构，导致方法的吞吐量较低。

为了改善这种效率瓶颈，可以选择摒弃 kNN 搜索，转而使用学生-教师模型，这是一种在深度学习中用于模型压缩和优化的方法，通过将复杂且性能优异的大教师模型的知识传递给较小的学生模型，使得学生模型在保持高准确度的同时，具有更低的计算开销。文献[13]使用了这种方法，根据文章的结果，由表 4-2 所示，最后两行是该论文中实现的应用知识蒸馏方法的两个变体，推理速度与吞吐量均高于 PatchCore 方法，证明了学生-教师模型在降低模型计算开销方面的潜力。

方法	图像级 AUROC	延迟(ms)	吞吐量(张/秒)
Simplenet	0.879	12	194
PatchCore	0.911	32	76
PatchCore(集成)	0.921	148	13
EfficientAD-S	0.954	2.2	614
EfficientAD-M	0.960	4.5	269

表 4-2 不同方法的性能

第五章 结论

在本报告中，我们使用 PatchCore 方法，在 MvTecAD_subset 数据集上进行图像级异常检测实验，通过对该方法的学习、实现、参数调优，最终，得到了图像级 AUROC 分数 0.98933 的成绩，表明我们使用的模型和方法可以有效的完成这个异常检测的任务。

这个任务使我收获颇丰，一开始使用的卷积网络+二分类的模型进行训练，得到的分数低于 0.5，表明一开始的模型性能甚至低于随机猜测。而后选择查阅论文，不断的尝试复现，终于找到了性能很好，并且也很好复现的 PatchCore 方法，然后在我们的任务上做了一些改造，达到了不错的成绩。

这次的任务对于我来说是第一次为了实现某个任务，而阅读论文，因此收获了相当的阅读论文的经验，使我对于 AI 领域的论文，尤其是会议论文的结构、阅读方法等方面有了更深入的理解。

参考文献

- [1]周锦珩.基于多模态大模型的工业异常检测方法研究与实现[D].上海师范大学,2025.
- [2]苏占智.基于机器视觉的工业制品缺陷检测算法研究[D].齐鲁工业大学,2025.
- [3]Bergman L, Cohen N, Hoshen Y. Deep nearest neighbor anomaly detection[J]. arXiv preprint arXiv:2002.10445, 2020.
- [4]Cohen N, Hoshen Y. Sub-image anomaly detection with deep pyramid correspondences[J]. arXiv preprint arXiv:2005.02357, 2020.
- [5] Defard T, Setkov A, Loesch A, et al. Padim: a patch distribution modeling framework for anomaly detection and localization[C]//International conference on pattern recognition. Cham: Springer International Publishing, 2021: 475-489.
- [6] Roth K, Pemula L, Zepeda J, et al. Towards total recall in industrial anomaly detection[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2022: 14318-14328.
- [7] Sener O, Savarese S. Active learning for convolutional neural networks: A core-set approach[J]. arXiv preprint arXiv:1708.00489, 2017.
- [8] Wolsey L A, Nemhauser G L. Integer and combinatorial optimization[M]. John

Wiley & Sons, 1999.

[9] Sener O, Savarese S. Active learning for convolutional neural networks: A core-set approach[J]. arXiv preprint arXiv:1708.00489, 2017.

[10] Sinha S, Zhang H, Goyal A, et al. Small-gan: Speeding up gan training using core-sets[C]//International Conference on Machine Learning. PMLR, 2020: 9005-9015.

[11] Liu Z, Zhou Y, Xu Y, et al. Simplenet: A simple network for image anomaly detection and localization[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2023: 20402-20411.

[12] Zhang X, Xu M, Zhou X. Realnet: A feature selection network with realistic synthetic anomaly for anomaly detection[C]//Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2024: 16699-16708.

[13] Batzner K, Heckler L, König R. Efficientad: Accurate visual anomaly detection at millisecond-level latencies[C]//Proceedings of the IEEE/CVF winter conference on applications of computer vision. 2024: 128-138.